AI-Powered Malicious Domain Analyser - Maintained Project Blueprint

1. Purpose

This repository implements a production-oriented cybercrime domain analysis pipeline that:

- ingests a list of suspicious domains or URLs,
- performs passive intelligence collection,
- captures browser evidence using Playwright desktop and mobile profiles,
- produces AI-assisted legal classification aligned to Indian cybercrime and financial-crime statutes,
- generates per-domain evidence reports and downloadable formal PDF drafts,
- publishes a consolidated dashboard to GitHub Pages, and
- preserves evidence packages as GitHub Actions artifacts.

This blueprint is the maintained source of truth for the current implementation. It supersedes prior PDF-only planning documents and incorporates all corrections made during repository hardening and subsequent feature work.

> Draft-handling note: formal report styling may reference NCTAU/I4C for workflow context, but generated reports remain analyst-produced drafts and must not be treated as official government issuances or takedown orders.

2. Current Deliverables

2.1 Inputs

- `input/domains.txt`
- optional `workflow_dispatch` text input `domains_input`
- repository secrets for all live API keys
- repo-tracked APNIC resource snapshot:
- `resources/APNIC_(IP&ASN)_Resources.csv`

2.2 Outputs

- per-domain JSON reports in `output/data/`
- per-domain HTML reports in `output/reports/`
- per-domain formal PDF reports in `output/reports/`

- per-domain evidence ZIP packages in `output/reports/`
- screenshots in `output/screenshots/`
- deployed static dashboard and per-domain pages in `docs/`
- workflow artifacts uploaded from `output/screenshots/`, `output/reports/`, and `output/data/`

3. Repository Structure

```
domain-analyser/
├── .github/workflows/
├── input/
├── output/
├── resources/
├── docs/
├── pipeline/
├── scripts/
├── templates/
├── LEGAL_USE.md
├── PROJECT_BLUEPRINT.md
├── PROJECT_BLUEPRINT.pdf
├── README.md
├── requirements.txt
├── run_local.py
└── setup.sh
```

4. Pipeline Phases

4.1 Phase 1 - Ingest

- Load domains from `input/domains.txt` or workflow text input.
- Normalize URLs and deduplicate entries.
- Calculate a lightweight priority score from naming signals and registration age heuristics.

4.2 Phase 2 - Passive Intelligence

Sources currently integrated:

- RDAP
- VirusTotal
- URLScan
- URLhaus
- Google Safe Browsing
- AlienVault OTX

- PhishTank
- AbuseIPDB
- DNS resolution
- crt.sh certificate transparency lookup
- APNIC resource allocation enrichment

Passive-intel calls must not stop the batch on single-source failure. Each source returns best-effort data and stores any failure under source-specific error fields.

4.3 APNIC Network Attribution

The APNIC enrichment layer uses the repo-tracked CSV snapshot and parses it once per run into three cached indexes:

- IPv4 allocation records
- IPv6 allocation records
- ASN records keyed by normalized holder name

For each domain, the system attempts attribution for:

- all resolved `A` records,
- all resolved `AAAA` records,
- `urlscan_page_ip` when present,
- `abuseipdb_ip` when present

Matching behavior:

- IPv4 and IPv6 use longest-prefix-match over APNIC allocation resources.
- The report stores matched allocations keyed by IP address.
- Holder-linked ASNs are derived by exact normalized holder-name matching against APNIC ASN rows.
- Holder-linked ASNs are explicitly presented as holder-linked allocations, not origin-AS assertions.

Returned network-attribution fields:

Field	Meaning
`resolved_ips`	unique IP set evaluated for attribution
`matched_allocations`	matched APNIC IPv4/IPv6 allocation rows, annotated with `ip_address`
`primary_holder`	dominant holder across matched allocations
`primary_cc`	country code from dominant matched allocation
`primary_economy_name`	economy/region from dominant matched allocation
`holder_linked_asns`	deduplicated ASN rows whose holder matches a matched allocation holder

4.4 Phase 3 - Active Capture

The browser capture stage:

- uses Playwright Chromium in headless mode,
- captures desktop and mobile Android views,
- applies stealth hardening,
- records page title, text, meta description, response headers, redirect chain, and network-request groupings,
- saves viewport and full-page PNG screenshots,
- computes SHA-256 hashes for each screenshot, and
- computes perceptual hashes for clustering.

4.5 Phase 4 - AI Legal Analysis

The AI stage:

- sends a structured prompt to Gemini,
- includes the India law reference block for IT Act 2000, BNS 2023, FEMA, PMLA, and related provisions,
- requests strict JSON output via `response_mime_type=application/json`,
- normalizes the result into the report contract, and
- falls back to a conservative heuristic classifier if Gemini is unavailable.

4.6 Phase 5 - Per-Domain Reporting

Each processed domain produces:

- raw JSON report
- interactive HTML report
- formal downloadable PDF report
- evidence manifest
- evidence ZIP package

The formal PDF report is rendered from the same report content using `weasyprint` with a print-optimized layout, page numbering, report identifier, and recurring draft disclaimer. When the host lacks the required WeasyPrint system libraries, the implementation falls back to Playwright PDF output so the report job still completes.

4.7 Phase 6 - Dashboard Publishing

The dashboard generation stage:

- rebuilds `docs/` from persisted report JSON,
- publishes per-domain HTML pages and PDF downloads,
- copies evidence ZIPs into the static site,
- emits `summary.json`, `domains.json`, and `domains.csv`,
- renders the dashboard UI using Jinja templates and vanilla JavaScript,

- downloads Chart.js with local fallback support, and
- deploys via GitHub Pages.

5. Report Contract

The persisted per-domain JSON includes at minimum:

- `domain`
- `input_url`
- `batch_id`
- `analysis_ts_utc`
- `registration`
- `dns_records`
- `cert_transparency`
- `threat_intel`
- `network_attribution`
- `captures`
- `ai_analysis`
- `evidence_manifest_hash`
- `report_identifier`
- `redirect_chain`
- `response_headers`
- `network_requests`
- `wayback_snapshot`
- `linked_domains`

5.1 Network Attribution Contract


```
{
  "resolved_ips": ["203.0.113.10"],
  "matched_allocations": [
    {
      "ip_address": "203.0.113.10",
      "ip_version": 4,
      "resource": "203.0.113.0/24",
      "start": "203.0.113.0",
      "value": "256",
      "nir": "irinn",
      "cc": "IN",
      "economy_name": "India",
      "delegation_date": "2026-04-08",
      "transfer_date": null,
      "opaque_id": "A1234567",
```

```
      "holder_name": "Example Telecom Pvt Ltd",
      "registry": "apnic",
      "type": "ipv4",
      "allocation_type": "ipv4"
    }
  ],
  "primary_holder": "Example Telecom Pvt Ltd",
  "primary_cc": "IN",
  "primary_economy_name": "India",
  "holder_linked_asns": [
    {
      "resource": "AS154594",
      "start": "154594",
      "value": "1",
      "nir": "",
      "cc": "IN",
      "economy_name": "India",
      "delegation_date": "2026-04-08",
      "transfer_date": null,
      "opaque_id": "A91AE317",
      "holder_name": "Example Telecom Pvt Ltd",
      "registry": "apnic",
      "type": "asn"
    }
  ]
}
```


Backward compatibility rule:

- older persisted reports that do not carry `network_attribution` must load successfully with empty/default attribution fields.

6. Per-Domain Report Requirements

Each interactive report page must expose:

- `Download PDF Report`
- `Download Evidence ZIP`
- `View Raw JSON`

Each report must contain:

1. Executive summary
2. Threat intelligence verdicts

3. Registration and DNS
4. Evidence capture
5. Legal analysis
6. Network attribution
7. Technical observations and link analysis
8. Evidence manifest and recommendations

Formal-report styling requirements:

- report identifier visible in header
- NCTAU/I4C draft context language
- mandatory analyst-draft disclaimer
- page numbering in PDF
- print-safe typography and spacing
- evidence screenshots and hashes preserved in the HTML source report

7. Dashboard Requirements

The dashboard must include:

- summary stat cards
- severity doughnut chart
- threat-category bar chart
- top hosting countries chart
- top impersonated brands table
- top allocation holders table
- infrastructure clusters table
- sortable/filterable/searchable domain table

The domain table must surface:

- domain
- severity
- category
- VT score
- registration date
- APNIC allocation holder
- APNIC region/economy
- status
- links to report, PDF, JSON, and ZIP

Infrastructure clustering dimensions:

- shared IP
- shared nameserver
- shared registrar

- shared visual template
- shared APNIC allocation holder

8. GitHub Actions and Deployment

8.1 Main Analysis Workflow

Workflow file:

- `.github/workflows/analyse.yml`

Trigger:

- `workflow_dispatch`

Inputs:

- `domains_input`
- `batch_name`
- `max_domains`

Current workflow hardening:

- uses `ubuntu-22.04` for Playwright compatibility with pinned dependencies
- configures GitHub Pages explicitly
- installs Python 3.11
- installs Playwright Chromium and browser dependencies
- installs WeasyPrint runtime libraries before report generation
- uploads artifacts with 90-day retention
- deploys `docs/` to GitHub Pages in the same job

8.2 Pages Requirements

Operational constraints discovered during implementation:

- GitHub Pages deployment through Actions requires the repository to support Pages for its visibility/plan combination.
- Repository secrets must be stored as GitHub Actions secrets, not in `.env.example`.
- The repo-tracked `.env.example` file must remain a blank template.

8.3 Secrets

Required secrets:

- `VT_API_KEY`
- `URLSCAN_API_KEY`
- `ABUSEIPDB_API_KEY`

- `OTX_API_KEY`
- `GOOGLE_SAFE_BROWSING_KEY`
- `GEMINI_API_KEY`

Optional:

- `HACKERTARGET_API_KEY`

9. Rate Limiting and Fault Tolerance

The shared async rate limiter remains mandatory.

Current caps:

- VirusTotal: 4 requests/minute
- URLScan: 10 requests/minute
- generic public API limiter: 30 requests/minute
- RDAP: 60 requests/minute
- Gemini: 15 requests/minute

Failure rules:

- any single domain may fail without stopping the batch
- any single source may fail without stopping a domain
- missing APNIC matches must never fail the domain

10. Local Operation

Default local entrypoint:

```
python run_local.py --batch-name batch-001
```

Local runs use the same pipeline components as GitHub Actions and write artifacts under `output/` and `docs/`.

Operational note:

- Ubuntu and other Debian-family environments should install Cairo, Pango, Harfbuzz, GDK-Pixbuf, and MIME support libraries before running PDF generation.
- The provided `setup.sh` now installs those packages automatically when `apt-get` is available.

11. Security and Evidence Handling

- `.env.example` is a template only and must never contain live secrets.
- If secrets are ever committed, rotate them and rewrite history before making the repository public.

- Evidence packages preserve HTML, PDF, raw JSON, manifest, and screenshots.
- SHA-256 hashes are recorded for screenshots, HTML, JSON, and PDF artifacts.

12. Blueprint Maintenance

Canonical blueprint source:

- `PROJECT_BLUEPRINT.md`

Generated distribution artifact:

- `PROJECT_BLUEPRINT.pdf`

Regeneration helper:

```
python scripts/render_blueprint_pdf.py
```

Any material workflow, report-contract, data-source, or deployment change must be reflected in both the Markdown source and the generated PDF.

13. Current Acceptance Checklist

- APNIC dataset is repo-tracked and available to GitHub Actions.
- Passive-intel output includes `network_attribution`.
- Report JSON remains backward compatible when older payloads are loaded.
- Each domain report page provides a downloadable PDF.
- Each evidence ZIP includes the PDF report.
- Dashboard surfaces APNIC holder/region context.
- Dashboard supports holder clustering.
- GitHub Actions still completes on `ubuntu-22.04`.
- GitHub Pages deployment remains operational.
- Blueprint Markdown and PDF exist in the repository and reflect the current system.